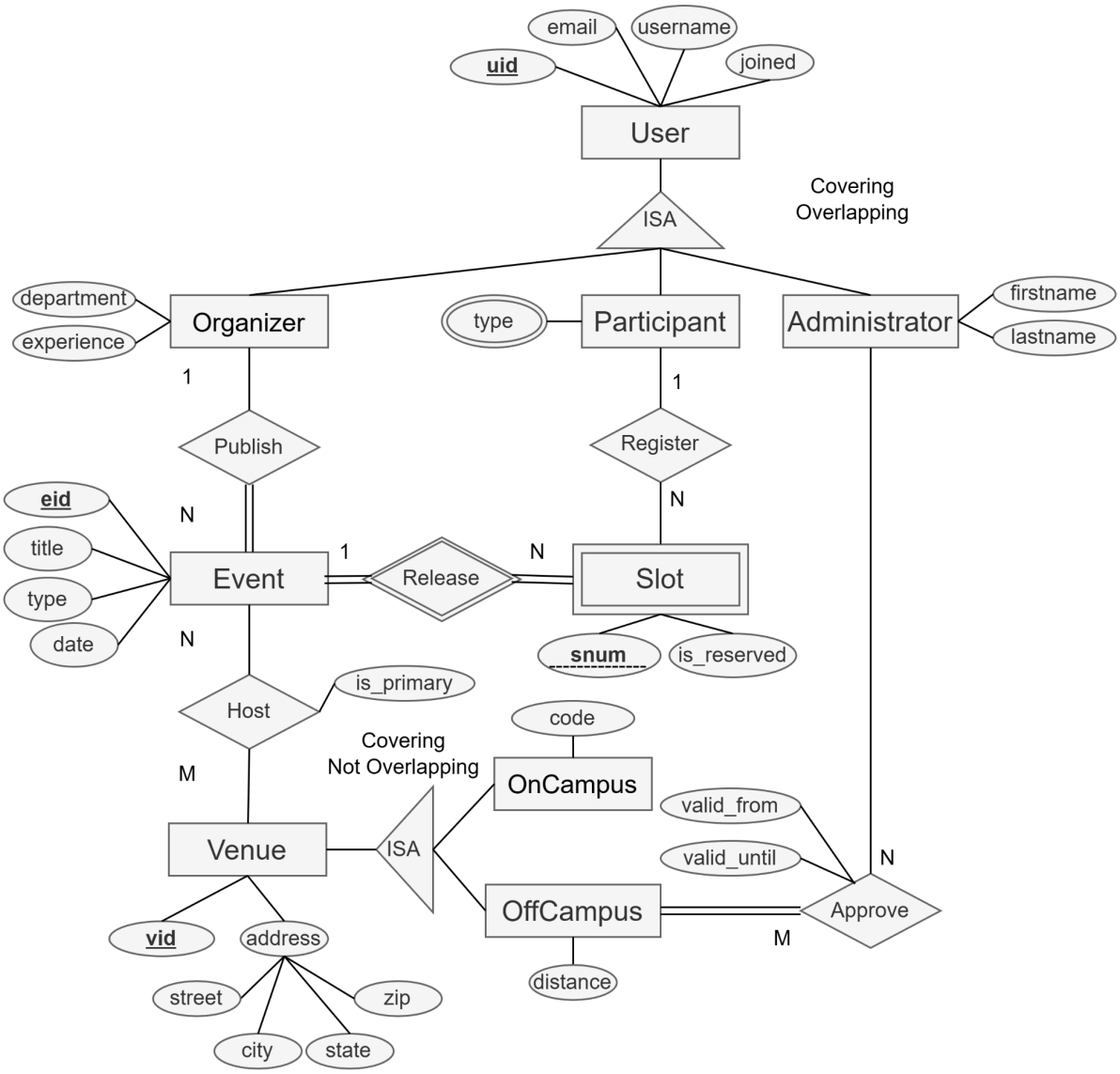
CS122A Project: Implement a Python program with MySQL

Introduction

In this project, you will implement a command-line program to manage the **ZotEvent** platform. You are required to use the Python MySQL connector to access and manipulate the database. The Python program should 1) accept command-line arguments as inputs, 2) parse the inputs into SQL statements, 3) execute the statements in the MySQL server, and 4) handle and print the results.

The ER diagram is shown as follows. Please refer to the HW2 solution for the DDLs**.**



Dataset

There will be one CSV file for each table. Each line represents one record, and the columns are separated by a comma ‘,’’. The order of columns follows the order of attributes in the DDL.

This **provided dataset** is for testing only, so you can make any changes to cover more cases. A hidden dataset will be used during the grading.

Regulations and Assumptions

1. You can have any number of Python files, but the entry/main file must be named “project.py”
2. The command to run the program will be

   python3 project.py <function name> [param1] [param2] …

   The list of function names and their parameters is in the function requirements section.
3. You can assume that the command-line input is always correct IN FORMAT ONLY. There won’t be a nonexistent function name as input, and the parameters will be given in the correct order and format. So you don’t need to handle unexpected input. However, input content can be faulty, e.g., given a duplicate netID for insertion.
4. You can assume that the dataset files are always correct IN FORMAT AND CONTENT. So there won’t be errors when parsing the file, or when inserting the records to DB.
5. Every date has the format YYYY-MM-DD, e.g. 2025-02-29, and every datetime has the format YYYY-MM-DD hh:mm:ss, e.g. 2025-02-29 14:10:34.
6. Strings that contain spaces will be wrapped in quotation marks when calling the command, (e.g. "The Matrix") whereas strings with no spaces will not have quotation marks (e.g. Wicked).
7. If the input is NULL, treat it as the None type in Python, not a string called “NULL”.
8. **If the output is boolean**, print “Success” or “Fail”.
9. **If the output is a result table**, print each record in one line and separate columns with ‘,’ - just like the format of the dataset file.
10. You must use **Python 3**. The standard Python libraries and mysql-connector-python will be installed in the autograder — other third-party packages are not allowed.

Function Requirements

1. Import data

Delete existing tables, and create new tables. Then read the csv files in the given folder and import data into the database. You can assume that the folder always contains all the necessary CSV files and the files are correct.

Input:

python3 project.py import [folderName:str]

python3 project.py import test_data

Output:

Boolean

2. Insert administrator

Insert a new user and administrator into the related tables.

Input:

python3 project.py insertAdmin [uid:int] [email:str] [username:str] [joined:date] [firstname:str] [lastname:str]

python3 project.py insertAdmin 1 admin@uci.edu awong 2024-04-19 Alice Wong

Output:

Boolean

3. Add venue to event

Add a venue to an existing event by inserting it into the Hosting relationship. The event and venue already exist. If is_primary = true, the function should ensure this event has no other primary venue. If another primary venue already exists, return False.

Input:

python3 project.py addVenue [eid:int] [vid:int] [is_primary:boolean]

python3 project.py addVenue 10 3 true

Output:

Boolean

4. Reserve slot

Reserve a specific slot for a participant. The event, slot, and participant already exist. The slot must currently be unreserved.

Input:

python3 project.py reserveSlot [eid:int] [snum:int] [uid:int]

python3 project.py reserveSlot 10 3 25

Output:

Boolean

5. Cancel reservation

Cancel a participant's reservation for a specific event slot. The function should mark the slot as unreserved and remove the participant from the slot. Only cancel if the slot is currently reserved by the given participant.

Input:

python3 project.py cancelReservation [eid:int] [snum:int] [uid:int]

python3 project.py cancelReservation 10 3 25

Output:

Boolean

6. **Update event**

   Update the title and the datetime of an event.

Input:

   python3 project.py updateEvent [eid:int] [title:str] [datetime:datetime]

   python3 project.py updateEvent 10 "Database Seminar" "2026-05-15 15:00:00"

Output:

   Boolean

7. **Delete organizer**

   Delete an organizer from the database. Deleting the organizer should also delete their events because of ON DELETE CASCADE. And deleting those events should also delete related slots and hosting records.

Input:

   python3 project.py deleteOrganizer [uid:int]

   python3 project.py deleteOrganizer 12

Output:

   Boolean

8. **Upcoming events with available slots**

   List all future events that still have at least one unreserved slot. Return event information along with the number of available slots. Sort by event datetime ascending, then event ID ascending.

Input:

   python3 project.py availableEvents [date:date]

   python3 project.py availableEvents 2026-05-01

Output:

   Table - eid, title, type, datetime, availableSlots

9. **Popular release**

   For each event type, compute the total number of reserved slots across all events of that type. Return only event types with at least N reserved slots. Sort by reserved count descending, then event type ascending.

Input:

   python3 project.py popularEventTypes [N:int]

   python3 project.py popularEventTypes 5

Output:

   Table - type, reservedCount

10. Participant schedule

Given a participant ID, list all events for which the participant has reserved a slot. Include the slot number and primary venue information if the event has a primary venue. Sort by event datetime ascending.

Input:

python3 project.py participantSchedule [uid:int]

python3 project.py participantSchedule 25

Output:

Table - eid, title, type, datetime, snum, vid, street, city, state, zip

11. Organizer event count

List organizers who have created at least N events. Sort by event count descending, then organizer UID ascending.

Input:

python3 project.py organizerStats [N:int]

python3 project.py organizerStats 3

Output:

Table - uid, username, department, eventCount

12. Venue event list

Given a venue ID, list all events hosted at that venue. Include whether the venue is primary for each event. Sort by event datetime ascending, then event ID ascending.

Input:

python3 project.py venueEvents [vid:int]

python3 project.py venueEvents 7

Output:

Table - eid, title, type, datetime, is_primary

Submission Notes

1. Submit a zip file (one submission per group) to Gradescope — remember to add your group members to the submission.

2. In the Gradescope autograder **(to be released soon)**, you need to connect the MySQL server with the following user, password, and database:

   mysql.connector.connect(user='test', password='password', database='cs122a')

   Make sure to change your code before submission if you are using other accounts in your local database.

3. The zip file should contain all the Python source files, including the entry point **project.py**. All files must be in the root level of the zip file, with no directories.

4. After you submit, the autograder will run public test cases with the public dataset we provided. Note that the displayed score is not your final grade; we will run private test cases after the project due date.